

DSPy

Programming — not prompting — language models

Learn how to stop hand-tuning prompt strings and start writing small, typed Python programs that a compiler improves for you. Every concept is explained from zero, illustrated with a diagram, and backed by runnable Python code.

Signatures

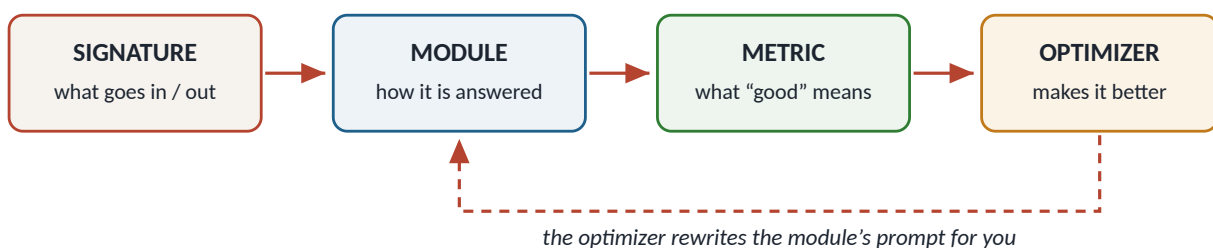
Modules

ReAct & tools

Metrics & evaluation

Optimizers (GEPA, MIPROv2)

A full worked project



Audience: Python developers who have called an LLM API at least once and have never used DSPy.

Covers: DSPy 3.x (verified against dspy 3.3.1, Python 3.10+).

Created: Wednesday, 9 September 2026 at 8:54 PM PDT

Prepared for: Sourav Bhattacharya

Contents

Chapter 1 — Why DSPy Exists	4
1.1 The problem with prompt strings	4
1.2 “Programming, not prompting”	4
1.3 The three things you will learn	5
1.4 What DSPy actually optimizes	6
1.5 When to use DSPy — and when not to	7
Chapter 2 — Setup and Your First Program	8
2.1 Installation	8
2.2 Pointing DSPy at a language model	8
2.3 Your first program	9
2.4 What just happened under the hood	10
2.5 Seeing the real prompt: <code>inspect_history</code>	10
2.6 Caching, cost and speed	10
2.7 A checklist before moving on	11
Chapter 3 — Signatures: Declaring What You Want	12
3.1 The idea	12
3.2 Inline (string) signatures	12
3.3 Class-based signatures	13
3.4 Richer output types	13
3.5 Editing signatures at runtime	14
3.6 How to write a good signature	15
Chapter 4 — Modules: Choosing How to Answer	16
4.1 The same task, three ways	16
4.2 The built-in modules	16
4.3 ChainOfThought in detail	17
4.4 ReAct and tools	18
4.5 Reliability wrappers: <code>BestOfN</code> and <code>Refine</code>	18
4.6 Running things in parallel	19
Chapter 5 — Composing Modules into Programs	20

5.1 The pattern	20
5.2 A worked pipeline: research assistant	20
5.3 A retriever, concretely	23
5.4 Inspecting a program	23
Chapter 6 — Data and Metrics: Defining “Better”	25
6.1 dspy.Example: one row of data	25
6.2 Metrics: turning judgement into a number	26
6.3 LM-as-judge metrics	26
6.4 Running an evaluation	27
6.5 Habits that will save you	28
Chapter 7 — Optimizers: Letting DSPy Write the Prompt	29
7.1 What an optimizer actually does	29
7.2 The three-line version	29
7.3 The optimizers worth knowing	30
7.4 BootstrapFewShot — the gentle start	30
7.5 MIPROv2 — the workhorse	31
7.6 GEPA — optimization that reads your feedback	31
7.7 Reading what the optimizer wrote	33
7.8 Practical rules	33
Chapter 8 — Saving, Debugging and Shipping	34
8.1 Saving state (the normal way)	34
8.2 Saving the whole program	34
8.3 Debugging	34
8.4 Serving it	35
8.5 A workflow that works	35
Chapter 9 — A Complete Worked Project	37
9.1 What we are building	37
9.2 Step 1 — signatures	37
9.3 Step 2 — the program	38
9.4 Step 3 — data and metric	39
9.5 Step 4 — baseline, optimize, compare	40
9.6 Step 5 — use it	41

9.7 What to try next on this project	41
Chapter 10 — Pitfalls, FAQ and Cheat Sheet	42
10.1 Ten beginner mistakes	42
10.2 Frequently asked questions	42
10.3 Cheat sheet	43
10.4 Where to go next	44
References — Sources and Further Reading	45

Why DSPy Exists

If you have ever written an LLM prompt, watched it work, changed one word, and watched it break — this chapter is about why that happens and what DSPy does instead.

1.1 The problem with prompt strings

The usual way to build an LLM feature is to write a big string:

THE WAY MOST PEOPLE START — A HAND-WRITTEN PROMPT

```
prompt = f"""You are an expert support agent. Read the customer email below and
return the category. Categories are: billing, technical, account, other.
Answer with ONLY the category word, no punctuation, no explanation.
Do not say anything else. Really, just one word.

Email: {email}
Category: """

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
)
category = response.choices[0].message.content.strip().lower().rstrip(".")
```

This works... until it doesn't. Four things are tangled together in that one string:

1. **The task** — an email goes in, a category comes out.
2. **The strategy** — answer directly, versus reason first, versus look something up.
3. **The wording** — “You are an expert...”, “ONLY the category word”, the pleading.
4. **The output plumbing** — `.strip().lower().rstrip(".")` and the regexes that follow.

Because they are tangled, you cannot change one without risking the others. Swap `gpt-4o-mini` for a cheaper local model and the careful wording that kept the old model in line stops working. Add a fifth category and you rewrite the sentence *and* the parser. Nobody on the team can tell you whether the last prompt edit made things better, because “better” was never written down as anything you could measure.

THE CORE IDEA

DSPy separates **what you want** from **how it is asked**, and then treats “how it is asked” as something a computer should figure out — not you.

1.2 “Programming, not prompting”

DSPy (Declarative Self-improving Python) is a Python framework from Stanford NLP. Its slogan is *the framework for programming — rather than prompting — language models*. In

DSPy you declare the shape of the task and let the framework write the actual prompt text. The same task above becomes three lines:

THE SAME TASK IN DSPY

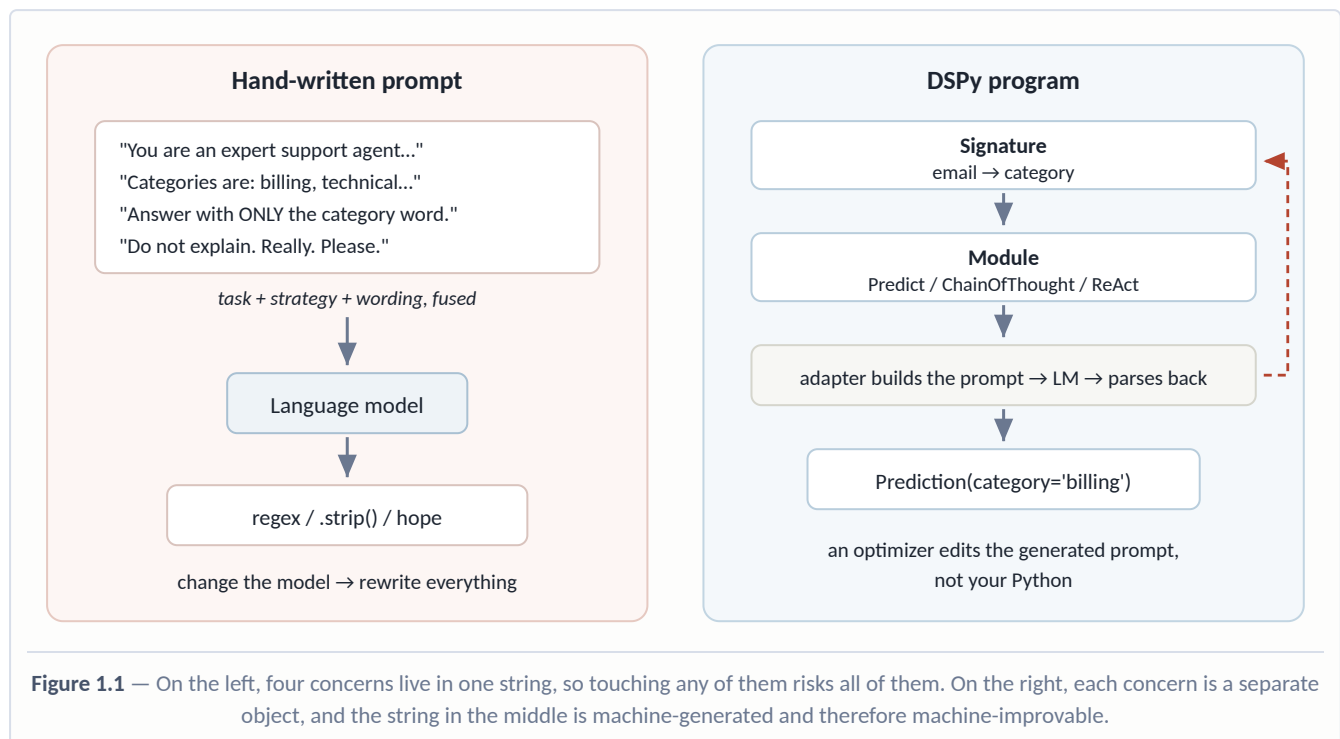
```
import dspy

dspy.configure(lm=dspy.LM("openai/gpt-4o-mini"))

classify = dspy.Predict("email -> category: Literal['billing', 'technical', 'account', 'other']")
result = classify(email=customer_email)

print(result.category)  # 'billing' -- already a clean, validated string
```

There is no prompt string anywhere in your codebase. DSPy builds one at call time from the declaration, sends it, parses the reply back into typed Python fields, and retries if the model returns something that doesn't fit. And — this is the part that matters later — because the prompt is generated rather than hand-written, a program can *rewrite it* for you.



1.3 The three things you will learn

Nearly all of DSPy is three ideas stacked on top of each other. The rest of this guide is one chapter per idea, plus a project that uses all three.

Concept	Answers the question	In one sentence
Signature	<i>What should happen?</i>	A typed declaration of the inputs and outputs of one LM call — like a function signature, e.g. <code>"email -> category"</code> .
Module	<i>How should it happen?</i>	A strategy for fulfilling a signature: answer directly, think step by step, use tools in a loop, sample several answers and pick the best.
Optimizer	<i>How do we make it better?</i>	An algorithm that, given examples and a metric, rewrites the instructions and picks the few-shot examples inside your modules.

A USEFUL ANALOGY

DSPy is deliberately shaped like PyTorch. A **Module** is a layer, a **program** made of modules is a network, a **metric** is your loss function, and an **optimizer** plays the role of the training loop. The difference is what gets learned: not floating-point weights, but the *text* of the instructions and the choice of few-shot examples.

1.4 What DSPy actually optimizes

This trips up almost every beginner, so let's be blunt. When you “compile” a DSPy program, **the model weights are not touched** (unless you deliberately choose a fine-tuning optimizer). What changes is:

- **The instruction text** attached to each signature — usually rewritten into something far more specific than what you wrote, because it was derived from your actual failures.
- **The few-shot demonstrations** embedded in the prompt — chosen from examples your own program got right, not invented by hand.

The output of optimization is therefore a small JSON file. You can read it, diff it in Git, and load it into a fresh program on another machine. That is the whole trick.

1.5 When to use DSPy — and when not to

Good fit	Poor fit
A task you will run thousands of times and want to be reliable	A one-off question you'll ask the model once
You have (or can write) 20-200 examples of good behaviour	You have zero examples and no way to judge output quality
You can define “correct” as a Python function, even loosely	Quality is purely a matter of taste with no proxy at all
Multi-step pipelines: extract → retrieve → reason → write	Free-form open-ended chat with a human in the loop
You want to move from a big model to a small cheap one	Latency budget forbids any extra LM calls at build time

REALITY CHECK

The single highest-value thing DSPy gives a beginner is not the optimizer. It is being *forced* to write down a metric and a handful of examples. Teams that do only that, and never call an optimizer, still ship better LLM features.

Setup and Your First Program

Ten minutes from an empty folder to a working, typed LLM call — and a look under the hood at the prompt DSPy wrote on your behalf.

2.1 Installation

DSPy requires **Python 3.10 or newer**. Install it into a virtual environment:

SHELL — INSTALL

```
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -U dspy

python -c "import dspy; print(dspy.__version__)"      # e.g. 3.3.1
```

2.2 Pointing DSPy at a language model

DSPy talks to models through **LiteLLM**, so one class covers every provider. You always write the model as `"provider/model-name"`.

CONFIGURING THE LM (PICK ONE)

```
import dspy

# OpenAI - reads OPENAI_API_KEY from the environment if api_key is omitted
lm = dspy.LM("openai/gpt-4o-mini", api_key="sk-...")

# Anthropic
lm = dspy.LM("anthropic/claude-sonnet-4-5", api_key="sk-ant-...")

# Google Gemini
lm = dspy.LM("gemini/gemini-2.5-flash", api_key="...")

# A local model served by Ollama - no API key, no bill
lm = dspy.LM("ollama_chat/llama3.2", api_base="http://localhost:11434", api_key="")

# Any OpenAI-compatible endpoint (vLLM, LM Studio, together, ...)
lm = dspy.LM("openai/my-model", api_base="http://localhost:8000/v1", api_key="local")

dspy.configure(lm=lm)      # set it once, globally, for the whole program
```

`dspy.configure` sets the default for everything that follows. When you need a different model for one section of code — a big model to judge, a small one to answer — use `dspy.context` instead of reconfiguring:

TEMPORARILY OVERRIDING THE MODEL

```
big = dsp.LM("openai/gpt-4o")

with dsp.context(lm=big):
    verdict = judge(question=q, answer=a)    # this call uses gpt-4o

routine = classify(email=e)                 # back to the global default
```

An `LM` object is also callable on its own, which is the quickest way to check that your keys work before any DSPy machinery is involved:

SMOKE TEST

```
print(lm("Reply with the single word: pong"))
# ['pong']
```

2.3 Your first program

Three lines. Note that we never write a prompt.

HELLO_DSPY.PY

```
import dsp

dsp.configure(lm=dsp.LM("openai/gpt-4o-mini"))

qa = dsp.Predict("question -> answer")

result = qa(question="Why is the sky blue at noon but red at sunset?")
print(result.answer)
```

OUTPUT (ABRIDGED)

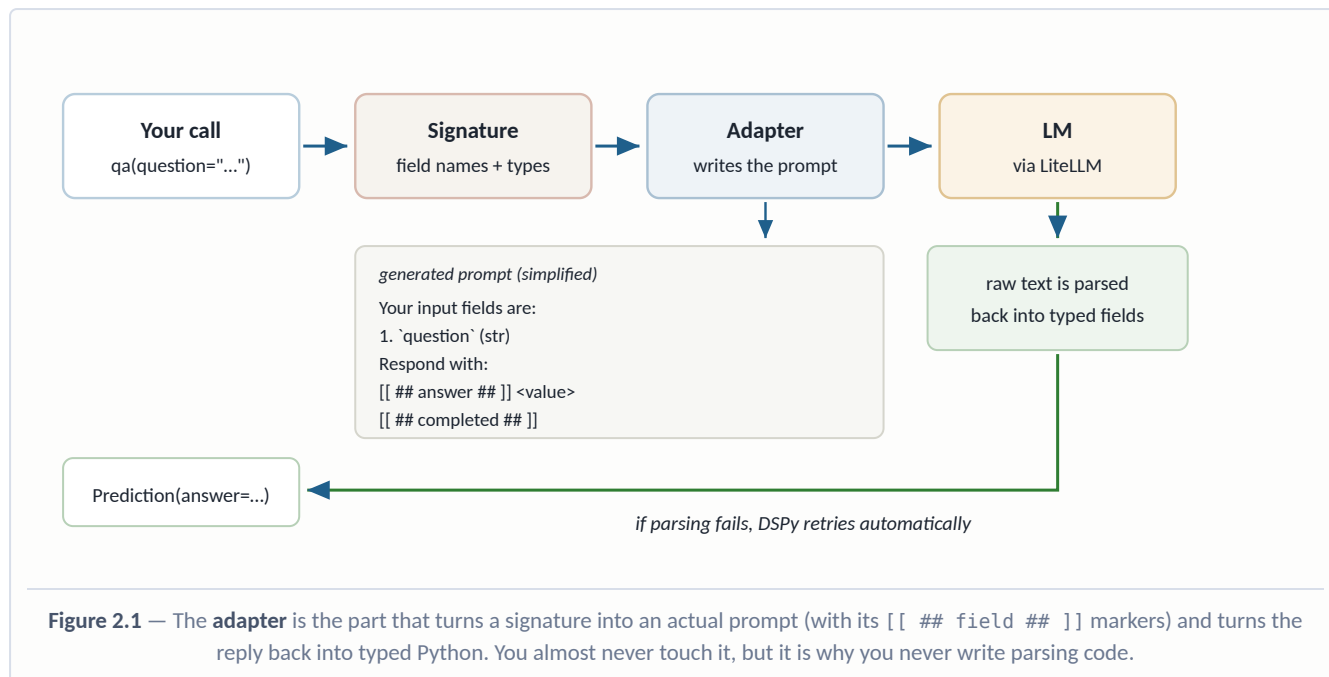
Sunlight contains all colours. Air molecules scatter short (blue) wavelengths far more strongly than long ones, so at noon the sky glows blue. At sunset the light travels a much longer path through the atmosphere, the blue is scattered away entirely, and mostly red and orange wavelengths survive to reach your eye.

Read the pieces:

- `"question -> answer"` is an **inline signature**. Everything left of the arrow is an input field; everything right of it is an output field. The names are not decoration — DSPy puts them in the prompt, so `question` and `answer` tell the model what those slots mean.
- `dsp.Predict(...)` wraps the signature in the simplest module: ask once, parse the reply.
- Calling it uses **keyword arguments matching the input field names**. Positional arguments will not work: `qa("why...")` raises an error.
- The return value is a `dsp.Prediction`. Access output fields as attributes: `result.answer`.

2.4 What just happened under the hood

Between your call and the model sit two invisible steps. Knowing they exist makes every later chapter obvious.



2.5 Seeing the real prompt: `inspect_history`

You should look at a generated prompt at least once. It removes all the mystery.

PRINT THE LAST LM INTERACTION

```
qa(question="Why is the sky blue?")

dspy.inspect_history(n=1)    # prints the exact system/user messages and the raw completion
```

DO THIS EARLY

When something behaves strangely — a field is empty, the model ignores an instruction —

`dspy.inspect_history(n=1)` is the first thing to run, every time. Ninety percent of beginner bugs are visible in that output immediately.

2.6 Caching, cost and speed

DSPy caches LM calls on disk and in memory by default, so re-running the same script does not re-bill you for identical calls. That is a gift during development and a trap during evaluation (you may measure a cache, not a model).

CONTROLLING THE CACHE

```
import dspy

# Turn caching off entirely (useful when measuring latency or sampling variety)
dspy.configure_cache(enable_disk_cache=False, enable_memory_cache=False)

# Or keep it, but ask a module for a fresh sample by raising temperature
creative = dspy.Predict("topic -> tagline", temperature=1.0)

# See what the last call cost you
print(lm.history[-1]["cost"], lm.history[-1]["usage"])
```

2.7 A checklist before moving on

You should be able to...	Which looks like
Install and import DSPy	<code>pip install -U dspy</code> ; <code>import dspy</code>
Configure any provider	<code>dspy.configure(lm=dspy.LM("provider/model"))</code>
Run a one-line program	<code>dspy.Predict("question -> answer")(question=q)</code>
Read the result	<code>result.answer</code> on the returned <code>Prediction</code>
See the generated prompt	<code>dspy.inspect_history(n=1)</code>

Signatures — Declaring What You Want

A signature is the single most important object in DSPy. Get comfortable here and everything else is a small step.

3.1 The idea

A **signature** declares the inputs and outputs of one LM call. It is the LLM equivalent of a Python function signature: it says *what* the call is for, never *how* to ask.

```
class TriageEmail(dspy.Signature):
    """Sort an inbound support email."""
    email: str = dspy.InputField()
    category: Category = dspy.OutputField()
    urgency: int = dspy.OutputField()
```

→ becomes the task instruction in the prompt
 → what you pass in
 → what you get back, type-checked & parsed

Field names are semantic, not cosmetic

The model literally reads the words “email”, “category”, “urgency”.

Renaming `x` → `y` to `email` → `category` changes the output quality. Choose real words.

Figure 3.1 — Every part of a class-based signature ends up somewhere in the generated prompt. Nothing is decorative.

3.2 Inline (string) signatures

For simple tasks a string is enough. The format is `"input_a, input_b -> output_a, output_b"`.

INLINE SIGNATURES OF INCREASING RICHNESS

```
import dspy
from typing import Literal

# 1. one in, one out
qa = dspy.Predict("question -> answer")

# 2. several inputs and several outputs
summarise = dspy.Predict("document, audience -> summary, key_terms")

# 3. add types after a colon — DSPy validates and coerces them for you
rate = dspy.Predict("review -> sentiment: float, is_spam: bool")

# 4. types can be containers or Literals
tag = dspy.Predict("article -> topics: list[str], tone: Literal['formal', 'casual']")

print(rate(review="Battery died in two days. Avoid.").sentiment) # -0.9 (a real float)
print(tag(article=text).topics) # ['fusion', 'energy policy']
```

UNTYPED MEANS STR

A field with no annotation is a `str`. So `"question -> answer"` is shorthand for `"question: str -> answer: str"`.

3.3 Class-based signatures

Reach for a class when you want a real instruction, per-field hints, or non-trivial types.

A CLASS-BASED SIGNATURE

```
import dspy
from typing import Literal

class TriageEmail(dspy.Signature):
    """Sort an inbound customer support email so it reaches the right team.

    Judge urgency by business impact, not by the customer's tone: an angry
    email about a cosmetic bug is low urgency; a calm email reporting that
    payments are failing is high urgency.
    """

    email: str = dspy.InputField(desc="Raw body text of the customer's email")
    account_tier: Literal["free", "pro", "enterprise"] = dspy.InputField()

    category: Literal["billing", "technical", "account", "other"] = dspy.OutputField()
    urgency: int = dspy.OutputField(desc="1 = can wait a week, 5 = wake someone up")
    one_line_summary: str = dspy.OutputField()

    triage = dspy.Predict(TriageEmail)
    r = triage(email=body, account_tier="pro")

    print(r.category, r.urgency, r.one_line_summary)
```

Four things are happening:

- The **docstring becomes the task instruction**. Write it as if briefing a competent new hire — especially the judgement calls, as above.
- `desc=` adds a per-field hint. Use it for units, ranges, and formats.
- `Literal[...]` constrains the output to a closed set. DSPy tells the model the allowed values and validates the reply against them.
- `urgency: int` comes back as a real Python `int`, so `r.urgency >= 4` just works — no casting, no stripping.

DOCSTRING VS DESC — AN IMPORTANT DIFFERENCE

Optimizers **rewrite the docstring** (the instruction) but leave `desc=` alone. So put stable, factual definitions in `desc=`, and put the strategy or emphasis you are unsure about in the docstring — that is the part DSPy is allowed to improve.

3.4 Richer output types

Any type that Pydantic understands can be an output field. This is how you get structured data out of an LLM without writing a JSON parser.

PYDANTIC MODELS AS OUTPUTS

```
import dspy
from pydantic import BaseModel, Field

class LineItem(BaseModel):
    description: str
    quantity: int
    unit_price: float

class Invoice(BaseModel):
    vendor: str
    invoice_number: str
    currency: str = Field(description="ISO-4217 code, e.g. USD")
    items: list[LineItem]
    total: float

class ExtractInvoice(dspy.Signature):
    """Extract structured invoice data from raw OCR text of a scanned document."""
    raw_text: str = dspy.InputField()
    invoice: Invoice = dspy.OutputField()

extract = dspy.Predict(ExtractInvoice)
inv = extract(raw_text=ocr_text).invoice

print(inv.total, len(inv.items), inv.items[0].description)  # real objects, already validated
```

Annotation	What DSPy does with it
<code>str</code>	Plain text (the default).
<code>int</code> , <code>float</code> , <code>bool</code>	Instructs the model to emit a bare value and parses it; retries on garbage.
<code>Literal["a", "b"]</code>	Lists the allowed values in the prompt and rejects anything else. The best choice for classification.
<code>list[str]</code> , <code>dict[str, int]</code>	Asks for JSON of that shape and parses it into real Python containers.
<code>Optional[X]</code>	Lets the model legitimately return nothing — useful for “field not present in document”.
A Pydantic <code>BaseModel</code>	Full nested schema in the prompt, validated on the way back. Field <code>description=</code> text is shown to the model.
<code>dspy.Image</code>	Passes an image to a multimodal model, e.g. <code>dspy.Image.from_file("chart.png")</code> .

3.5 Editing signatures at runtime

Signatures are objects, so you can derive new ones without redefining the class. This is handy for A/B testing an instruction, and it is exactly what optimizers do internally.

DERIVING NEW SIGNATURES

```
sig = dspy.Signature("email -> category")

sig2 = sig.with_instructions("Classify the email. Prefer 'billing' whenever money is mentioned.")
sig3 = sig2.append("confidence", dspy.OutputField(desc="0.0-1.0"), type_=float)

clf = dspy.Predict(sig3)
out = clf(email=body)
print(out.category, out.confidence)
```

3.6 How to write a good signature

Do	Don't
Name fields with real words: <code>customer_email</code> , <code>refund_reason</code>	Use <code>input1</code> , <code>text</code> , <code>x</code>
Split one call into several output fields when the parts are distinct	Ask for one blob and regex it apart afterwards
Use <code>Literal</code> for anything with a fixed vocabulary	Ask for a free-text label and normalise it yourself
Put the <i>judgement rule</i> in the docstring	Put a whole essay of formatting rules there — the adapter handles format
Keep each signature to one job	Build a mega-signature with fourteen outputs; compose modules instead (Chapter 5)

DON'T WRITE FORMATTING INSTRUCTIONS

Never write “respond in JSON” or “output only the answer” in a DSPy docstring. The adapter already enforces the output format, and your extra rules fight with it. Describe the *task*, not the *shape*.

Modules — Choosing How to Answer

One signature, many strategies. Swapping a module is a one-word change that can move accuracy more than a day of prompt tinkering.

4.1 The same task, three ways

A module takes a signature and decides *how* to fulfil it. Because the signature is separate, you can trade strategies freely:

SWAPPING STRATEGIES WITHOUT TOUCHING THE TASK

```
sig = "question -> answer"

fast      = dspy.Predict(sig)           # ask once
thinking  = dspy.ChainOfThought(sig)    # reason first, then answer
agent     = dspy.ReAct(sig, tools=[web_search]) # look things up in a loop

for m in (fast, thinking, agent):
    print(m(question="How many time zones does Russia have today?").answer)
```

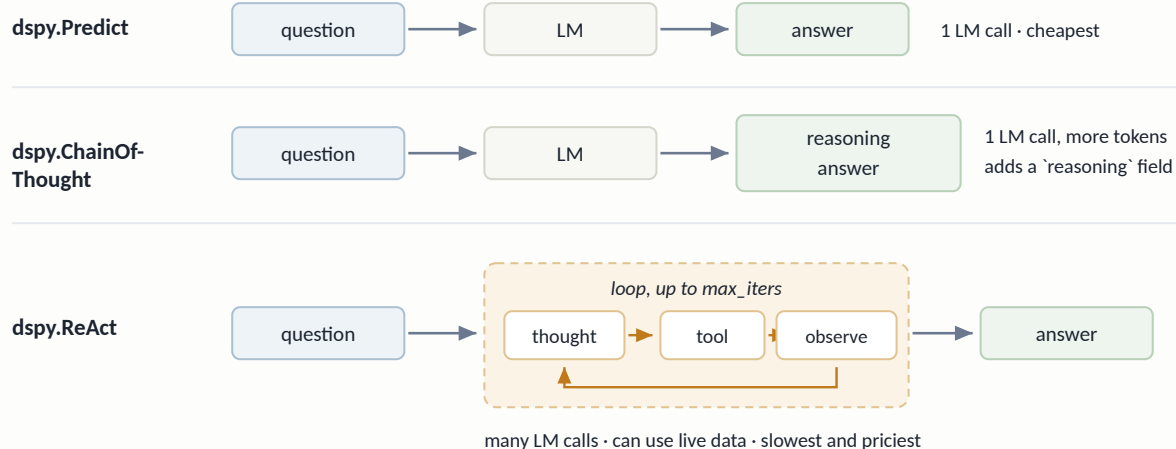


Figure 4.1 — The signature `question -> answer` is identical in all three rows. Only the module changes, and with it the cost, latency and capability.

4.2 The built-in modules

Module	What it does	Reach for it when...
<code>dspy.Predict</code>	One call, straight to the answer.	The task is simple, or you are establishing a baseline. Always start here.

Module	What it does	Reach for it when...
<code>dspy.ChainOfThought</code>	Adds a <code>reasoning</code> output field so the model thinks before answering.	Anything involving arithmetic, multi-fact comparison, or judgement.
<code>dspy.ReAct</code>	Interleaves thoughts with tool calls until it finishes.	The answer depends on data the model doesn't have: search, a database, an API.
<code>dspy.ProgramOfThought</code>	Writes Python, executes it, and answers from the result.	Exact arithmetic, date maths, data manipulation.
<code>dspy.CodeAct</code>	An agent whose actions are Python code that calls your functions.	Tool use where the tools need to be combined and looped over.
<code>dspy.BestOfN</code>	Runs a module N times and keeps the highest-scoring result.	You have a cheap reward function and can afford N× the cost.
<code>dspy.Refine</code>	Like BestOfN, but feeds critique of each attempt into the next.	Output must satisfy hard constraints (length, style, no forbidden words).
<code>dspy.Parallel</code>	Runs many independent calls concurrently.	Batch processing — classifying 5,000 emails.
<code>dspy.MultiChainComparison</code>	Compares several reasoning chains and synthesises a final answer.	High-stakes questions where one chain is fragile.
<code>dspy.Module</code>	The base class you subclass to build your own.	Any pipeline of more than one step — see Chapter 5.

4.3 ChainOfThought in detail

`ChainOfThought` silently adds one extra output field, `reasoning`, before your own outputs. That is the entire mechanism — and it is often worth several accuracy points.

READING THE REASONING

```
solve = dspy.ChainOfThought("word_problem -> answer: float")

r = solve(word_problem="A train leaves at 09:40 and arrives at 13:15. "
            "It stopped for 25 minutes. How many hours was it moving?")

print(r.reasoning)  # "From 09:40 to 13:15 is 3 hours 35 minutes = 3.583 h. Subtract the ..."
print(r.answer)    # 3.1666...
```

WHY NOT USE IT EVERYWHERE?

It costs more tokens and latency, and on trivially easy tasks it can talk itself out of a correct answer. The honest workflow is: build with `Predict`, measure, then try `ChainOfThought` and measure again (Chapter 6 shows how).

4.4 ReAct and tools

A tool in DSPy is just a Python function. The function's **name**, **type hints** and **docstring** are what the model sees, so write them for a reader.

AN AGENT WITH TWO TOOLS

```
import dspy

def search_docs(query: str) -> list[str]:
    """Search the internal product documentation and return matching passages."""
    return vector_store.similarity_search(query, k=4)

def get_order_status(order_id: str) -> str:
    """Look up the current shipping status of an order by its ID, e.g. 'A-11823'."""
    return orders_api.status(order_id)

agent = dspy.ReAct(
    "customer_question -> reply",
    tools=[search_docs, get_order_status],
    max_iters=6,          # hard cap on the think/act loop; default is 20
)

out = agent(customer_question="Where is order A-11823 and can I still change the address?")

print(out.reply)
print(out.trajectory)    # {'thought_0': ..., 'tool_name_0': 'get_order_status', ...}
```

The returned `trajectory` dictionary is your audit log. For each step `i` it contains `thought_i`, `tool_name_i`, `tool_args_i` and `observation_i`. Print it whenever an agent misbehaves.

TOOL DOCSTRINGS ARE PROMPTS

“Look up the current shipping status of an order by its ID, e.g. 'A-11823'” performs far better than “gets status”. If your agent keeps calling the wrong tool, fix the docstrings before you touch anything else.

4.5 Reliability wrappers: BestOfN and Refine

SAMPLING UNTIL A CONSTRAINT IS MET

```
import dspy

def not_too_long(args, pred) -> float:
    """Reward short taglines: 1.0 if under 60 characters, else 0.0."""
    return 1.0 if len(pred.tagline) < 60 else 0.0

base = dspy.ChainOfThought("product_description -> tagline")
best = dspy.BestOfN(module=base, N=5, reward_fn=not_too_long, threshold=1.0)
refine = dspy.Refine(module=base, N=5, reward_fn=not_too_long, threshold=1.0)

print(best(product_description=desc).tagline)
```

`BestOfN` samples `N` independent attempts (each at a different temperature) and returns the first that clears `threshold`, or the best one seen. `Refine` does the same but shows the model what was wrong with the previous attempt, so it tends to succeed in fewer tries.

4.6 Running things in parallel

CLASSIFYING A BATCH CONCURRENTLY

```
classify = dspy.Predict("email -> category: Literal['billing','technical','account','other']")

runner = dspy.Parallel(num_threads=16)
batch = [(classify, {"email": e}) for e in emails]      # (module, kwargs) pairs
results = runner(batch)

print([r.category for r in results[:5]])
```

RULE OF THUMB FOR PICKING A MODULE

Does the answer need **information you don't have**? → `ReAct` .

Does it need **exact computation**? → `ProgramOfThought` .

Does it need **careful thinking**? → `ChainOfThought` .

Otherwise → `Predict` . Then measure, and only then upgrade.

Composing Modules into Programs

Real systems are pipelines. In DSPy a pipeline is an ordinary Python class — and the optimizer can tune every step inside it at once.

5.1 The pattern

Subclass `dspy.Module`, create your sub-modules in `__init__`, and wire them together in `forward`. If you know PyTorch, this is muscle memory already.

THE SHAPE OF EVERY DSPY PROGRAM

```
import dspy

class MyProgram(dspy.Module):
    def __init__(self):
        super().__init__()          # always call this
        self.step_one = dspy.ChainOfThought("a -> b")
        self.step_two = dspy.Predict("b, a -> c")

    def forward(self, a: str) -> dspy.Prediction:
        b = self.step_one(a=a).b      # plain Python between the steps
        out = self.step_two(a=a, b=b)
        return dspy.Prediction(b=b, c=out.c)  # return whatever the caller needs

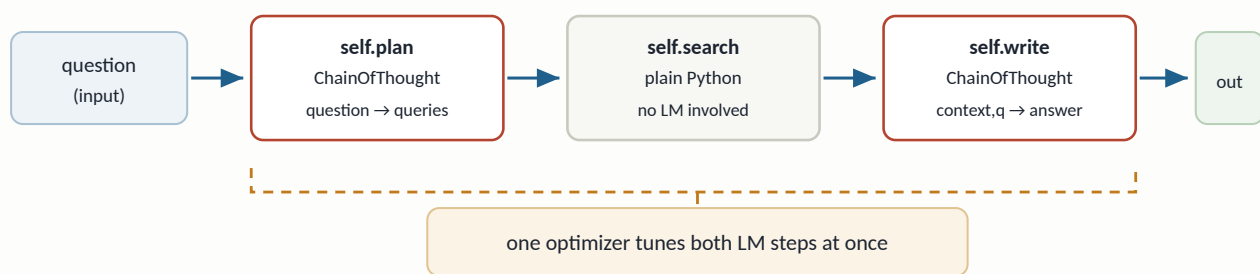
program = MyProgram()
print(program(a="hello").c)          # __call__ routes to forward()
```

WHY THIS MATTERS

Because the sub-modules are attributes of the class, DSPy can walk your program, find every prompt inside it, and optimize *all of them together* against one end-to-end metric — even though you only ever score the final output.

5.2 A worked pipeline: research assistant

Three steps: turn a question into search queries, retrieve, then answer with citations and a self-assessed confidence.



you only score the final answer — DSPy figures out the intermediate steps

Figure 5.1 — A DSPy program mixes LM steps with ordinary Python. Only the LM steps carry prompts, and those are the ones an optimizer will rewrite.

```

import dspy
from typing import Literal

class PlanSearches(dspy.Signature):
    """Turn a research question into 2-4 focused search queries.

    Each query should target a different facet of the question so the
    combined results cover it fully, rather than repeating one angle.
    """
    question: str = dspy.InputField()
    queries: list[str] = dspy.OutputField()

class AnswerWithSources(dspy.Signature):
    """Answer the question using only the supplied context passages.

    If the context does not contain the answer, say so plainly rather
    than guessing. Cite passages by their number, like [2].
    """
    question: str = dspy.InputField()
    context: list[str] = dspy.InputField()
    answer: str = dspy.OutputField()
    confidence: Literal["high", "medium", "low"] = dspy.OutputField()

class ResearchAssistant(dspy.Module):
    def __init__(self, retriever, max_passages: int = 6):
        super().__init__()
        self.retriever = retriever # any callable: str -> list[str]
        self.max_passages = max_passages
        self.plan = dspy.ChainOfThought(PlanSearches)
        self.write = dspy.ChainOfThought(AnswerWithSources)

    def forward(self, question: str) -> dspy.Prediction:
        queries = self.plan(question=question).queries

        passages, seen = [], set()
        for q in queries: # ordinary Python, no magic
            for p in self.retriever(q):
                if p not in seen:
                    seen.add(p)
                    passages.append(p)
        passages = passages[: self.max_passages]

        out = self.write(question=question, context=passages)

        return dspy.Prediction(
            answer=out.answer,
            confidence=out.confidence,
            queries=queries,
            context=passages,
        )

```

USING IT

```
dspy.configure(lm=dspy.LM("openai/gpt-4o-mini"))

assistant = ResearchAssistant(retriever=my_vector_search)
r = assistant(question="What caused the 2021 semiconductor shortage?")

print(r.answer)
print(r.confidence, "|", r.queries)
```

5.3 A retriever, concretely

DSPy ships a simple embeddings retriever so you can build a RAG demo with no external database.

RETRIEVAL-AUGMENTED GENERATION IN TEN LINES

```
import dspy

corpus = [
    "DSPy signatures declare the inputs and outputs of an LM call.",
    "Modules such as Predict and ChainOfThought decide how a signature is executed.",
    "Optimizers rewrite instructions and select few-shot demonstrations.",
    # ... thousands more chunks
]

embedder = dspy.Embedder("openai/text-embedding-3-small", dimensions=512)
search = dspy.retrievers.Embeddings(embedder=embedder, corpus=corpus, k=5)

class RAG(dspy.Module):
    def __init__(self):
        super().__init__()
        self.respond = dspy.ChainOfThought("context, question -> response")

    def forward(self, question):
        context = search(question).passages
        return self.respond(context=context, question=question)

print(RAG()(question="What does an optimizer change?").response)
```

5.4 Inspecting a program

Two methods repay learning early. `named_predictors()` lists every prompt-bearing step DSPy found; `inspect_history()` shows what each one actually sent.

WHAT IS INSIDE MY PROGRAM?

```
assistant = ResearchAssistant(retriever=my_vector_search)

for name, predictor in assistant.named_predictors():
    print(name, "|", predictor.signature.instructions[:70], "...")

# plan.predict      | Turn a research question into 2-4 focused search queries. ...
# write.predict     | Answer the question using only the supplied context passages. ...

assistant(question="...")
dspy.inspect_history(n=2)      # both LM calls, in order
```

THREE MISTAKES PEOPLE MAKE IN FORWARD

- Forgetting `super().__init__()` — the optimizer then finds no sub-modules.
- Creating modules inside `forward` instead of `__init__` — a fresh, unoptimized module is built on every call, so optimization has nothing to attach to.
- Returning a bare string. Return a `dspy.Prediction`, so metrics and optimizers can read named fields.

Data and Metrics — Defining “Better”

Optimization is impossible without a score. This chapter is short, and it is the one that changes how you work.

6.1 dspy.Example: one row of data

An `Example` is a small typed record. It holds both the inputs your program will receive and, usually, the answer you expect. `.with_inputs(...)` marks which fields are inputs — everything else is treated as a label.

BUILDING A DATASET

```
import dspy

ex = dspy.Example(
    question="What is the capital of Australia?",
    answer="Canberra",
).with_inputs("question")      # 'question' goes in; 'answer' is the label

print(ex.inputs())             # Example(question='What is the capital of Australia?')
print(ex.labels())             # Example(answer='Canberra')
print(ex.question, ex.answer)  # attribute access, like a namedtuple
```

FROM A JSONL FILE, THEN SPLIT

```
import json, random

rows = [json.loads(line) for line in open("support_tickets.jsonl")]

data = [
    dspy.Example(email=r["email"], account_tier=r["tier"],
                  category=r["category"], urgency=r["urgency"])
    .with_inputs("email", "account_tier")
    for r in rows
]

random.Random(0).shuffle(data)          # fixed seed = reproducible splits
n = len(data)
trainset = data[:int(0.5 * n)]          # optimizer learns from these
valset    = data[int(0.5 * n) : int(0.75 * n)] # optimizer selects using these
testset   = data[int(0.75 * n) :]       # you report on these, once
```

HOW MUCH DATA DO I NEED?

Far less than you fear. **20 examples** is enough to start optimizing; **50-200** is a comfortable range; **300+** mostly buys you a more trustworthy measurement rather than a better program. Examples need inputs; labels are optional if your metric doesn't need them.

6.2 Metrics: turning judgement into a number

A metric is a plain function `metric(example, prediction, trace=None)` returning a number (or a bool). Higher is better; scores in `[0.0, 1.0]` are conventional.

THREE METRICS, FROM STRICT TO NUANCED

```
# 1. Exact match – perfect for Literal outputs
def category_match(example, prediction, trace=None) -> float:
    return float(example.category == prediction.category)

# 2. Partial credit – combine several criteria with weights
def triage_quality(example, prediction, trace=None) -> float:
    correct_cat = example.category == prediction.category
    close_urgency = abs(example.urgency - prediction.urgency) <= 1
    short_summary = len(prediction.one_line_summary.split()) <= 20
    return 0.6 * correct_cat + 0.3 * close_urgency + 0.1 * short_summary

# 3. Recall of required key points
def covers_key_points(example, prediction, trace=None) -> float:
    text = prediction.answer.lower()
    hits = sum(point.lower() in text for point in example.must_mention)
    return hits / len(example.must_mention)
```

WHAT IS TRACE FOR?

During optimization DSPy passes the execution trace as a third argument. The convention is: when `trace is not None`, return a strict `bool` (“is this example good enough to become a demonstration?”); when it is `None` you are being evaluated, so return the graded float. You can ignore it entirely at first.

6.3 LM-as-judge metrics

For open-ended output there is no string to compare against. Score it with another DSPy program — ideally a stronger model.

A JUDGE IS JUST ANOTHER DSPY PROGRAM

```
import dspy

class GradeAnswer(dspy.Signature):
    """Grade a candidate answer against a reference answer.

    Judge factual agreement only. Differences of wording, ordering or
    length do not matter. An answer that adds correct extra detail is fine;
    one that adds a claim absent from the reference is not.
    """
    question: str = dspy.InputField()
    reference: str = dspy.InputField()
    candidate: str = dspy.InputField()
    faithful: bool = dspy.OutputField()
    score: float = dspy.OutputField(desc="0.0 to 1.0")

_judge_lm = dspy.LM("openai/gpt-4o")          # judge with a stronger model than you answer with
_judge = dspy.ChainOfThought(GradeAnswer)

def judged(example, prediction, trace=None) -> float:
    with dspy.context(lm=_judge_lm):
        g = _judge(question=example.question,
                    reference=example.answer,
                    candidate=prediction.answer)
    return float(g.score) if g.faithful else 0.0
```

DSPy also ships ready-made ones, e.g. `dspy.evaluate.SemanticF1`, which scores the overlap of factual claims between a prediction and a reference:

A BUILT-IN SEMANTIC METRIC

```
from dspy.evaluate import SemanticF1

metric = SemanticF1(decompositional=True)
```

6.4 Running an evaluation

MEASURING A PROGRAM

```
evaluate = dspy.Evaluate(
    devset=valset,
    metric=trriage_quality,
    num_threads=16,          # parallel – a 200-example run takes seconds, not minutes
    display_progress=True,
    display_table=5,         # show the 5 worst/first rows inline
)

result = evaluate(my_program)
print(result.score)          # e.g. 71.4 (a percentage)

for example, prediction, score in result.results[:3]:
    print(score, "|", example.email[:40], "->", prediction.category)
```



an optimizer is exactly this loop, run many times, keeping whatever scores highest

Figure 6.1 — Evaluation is the foundation everything else stands on. Build this before you build anything clever.

6.5 Habits that will save you

Habit	Why
Write the metric before the program	It forces you to say what “good” means while you can still change the design.
Keep a test set you evaluate on <i>once</i>	Optimizing against the same set you report on flatters you into shipping something worse.
Look at the ten worst rows by hand	Most metric bugs and most data bugs are visible in ten rows.
Start with an over-strict metric	You can always relax it; a lenient metric hides regressions from day one.
Fix your shuffle seed	Otherwise yesterday's number and today's aren't comparable.

Optimizers — Letting DSPy Write the Prompt

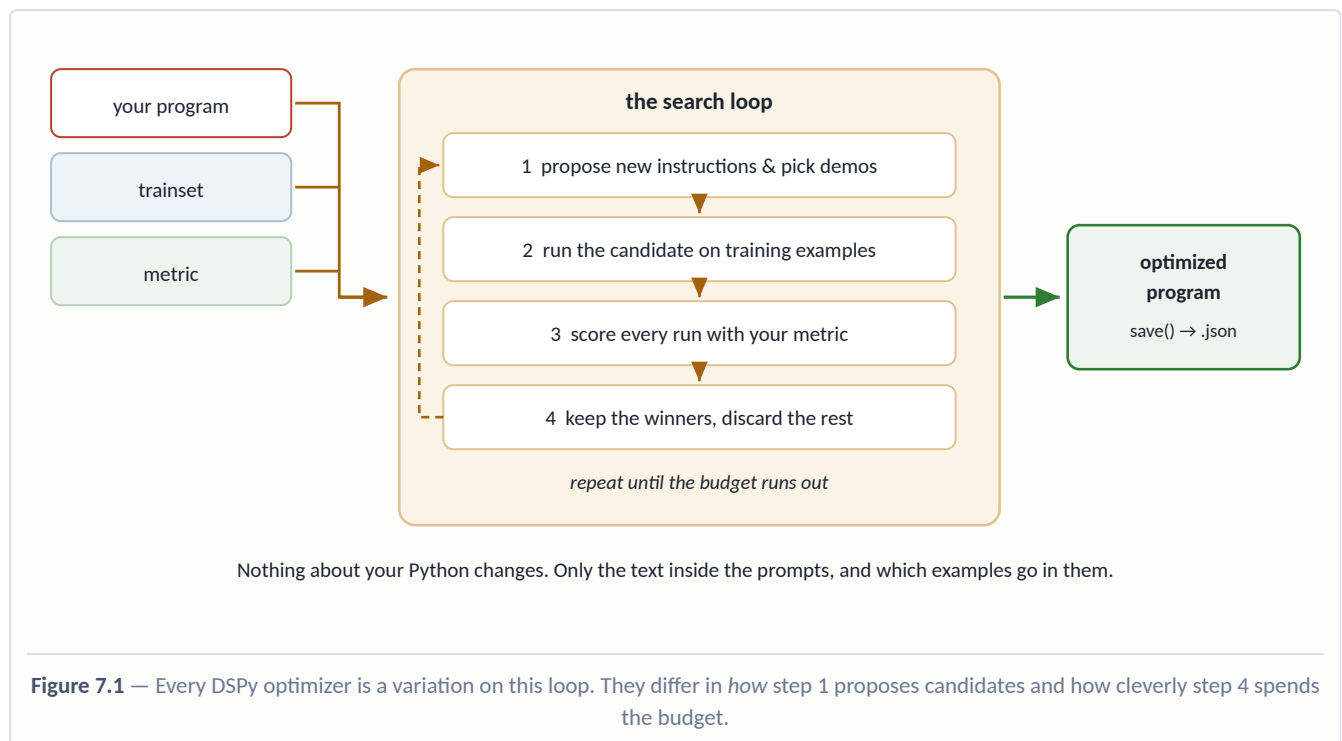
This is the payoff chapter. You supply a program, ~30 examples and a metric; DSPy supplies a better prompt than you would have written.

7.1 What an optimizer actually does

An optimizer (historically called a *teleprompter*) takes three things and returns a new, improved copy of your program:

1. your **program** (any `dspy.Module`),
2. a **trainset** of `dspy.Example`s,
3. a **metric**.

It then searches over two things inside your program: the **instruction text** of each signature, and the **few-shot demonstrations** attached to each predictor. Your Python is never modified, and neither are the model's weights.



7.2 The three-line version

OPTIMIZING ANY PROGRAM

```
optimizer = dspy.MIPROv2(metric=triage_quality, auto="light")
optimized = optimizer.compile(my_program, trainset=trainset)

optimized.save("triage_v1.json")
```

That is genuinely the whole API. `compile()` returns a *new* program; your original is untouched, so you can compare them side by side.

7.3 The optimizers worth knowing

Optimizer	Needs	How it works	Use when
<code>LabeledFewShot</code>	~5-20 ex.	Drops your labelled examples straight into the prompt.	A zero-cost sanity baseline.
<code>BootstrapFewShot</code>	~10-50 ex.	Runs your program, keeps runs the metric approved, and reuses them as demonstrations.	Your first real optimization. Cheap and fast.
<code>BootstrapFewShot- WithRandomSearch</code>	~50 ex.	The above, repeated with different random demo sets; keeps the best.	You have a bit more budget and want a solid gain.
<code>KNNFewShot</code>	~50+ ex.	Picks demonstrations at <i>run</i> time by embedding similarity to the input.	Inputs vary a lot in kind or domain.
<code>COPRO</code>	~30+ ex.	Iteratively rewrites the instruction text via coordinate ascent.	Wording is the bottleneck, not examples.
<code>MIPROv2</code>	~50-200 ex.	Bayesian search over instructions <i>and</i> demo sets jointly. The reliable workhorse.	The default choice for most tasks.
<code>GEPA</code>	~20-100 ex.	A reflective loop: a strong model reads your failures <i>and your textual feedback</i> and rewrites instructions.	You can explain <i>why</i> an answer is wrong in words. Often the strongest option.
<code>SIMBA</code>	~50+ ex.	Mini-batch stochastic search that introspects its own trajectories.	Larger programs, bigger budgets.
<code>BootstrapFinetune</code>	~300+ ex.	Distils a prompted program into actual fine-tuned weights.	Moving a proven pipeline onto a small local model.
<code>BetterTogether</code>	~300+ ex.	Alternates prompt optimization and fine-tuning.	Squeezing the last few points out.
<code>Ensemble</code>	—	Combines several compiled programs and aggregates their answers.	Accuracy matters more than cost.

7.4 BootstrapFewShot — the gentle start

The insight is charming: instead of writing examples by hand, run your own program on the training inputs, keep the runs your metric scored well, and paste *those* into the prompt as

demonstrations. Self-generated, self-verified few-shot learning.

YOUR FIRST OPTIMIZATION RUN

```
optimizer = dspy.BootstrapFewShot(
    metric=category_match,
    max_bootstrapped_demos=4,      # self-generated examples to include
    max_labeled_demos=8,          # raw labelled examples to include
    max_rounds=1,
)

optimized = optimizer.compile(student=trriage_program, trainset=trainset)

before = dspy.Evaluate(devset=testset, metric=category_match, num_threads=16)(trriage_program)
after  = dspy.Evaluate(devset=testset, metric=category_match, num_threads=16)(optimized)
print(f"before.score:.1f}% -> {after.score:.1f}%")
```

7.5 MIPROv2 — the workhorse

MIPROv2 searches instructions and demonstrations together using Bayesian optimization. The `auto` parameter picks a budget for you.

MIPROV2

```
optimizer = dspy.MIPROv2(
    metric=trriage_quality,
    auto="medium",      # "light" (fast, cheap) | "medium" | "heavy" (slow, best)
    num_threads=16,
)

optimized = optimizer.compile(
    triage_program,
    trainset=trainset,
    valset=valset,      # used to select the winning candidate
    max_bootstrapped_demos=2,
    max_labeled_demos=2,
)
```

BUDGET GUIDANCE

Start with `auto="light"`. It usually takes a few minutes and a few cents, and it tells you whether optimization helps this task at all. Only spend on `"heavy"` once `"light"` has shown a gain.

7.6 GEPA — optimization that reads your feedback

GEPA (Genetic-Pareto) is different in one crucial way: your metric may return **text** alongside the score. A strong “reflection” model reads that feedback together with the failing example and proposes a better instruction — the way a good mentor would.

ordinary optimizers see only a number; GEPA also sees **why** it was wrong

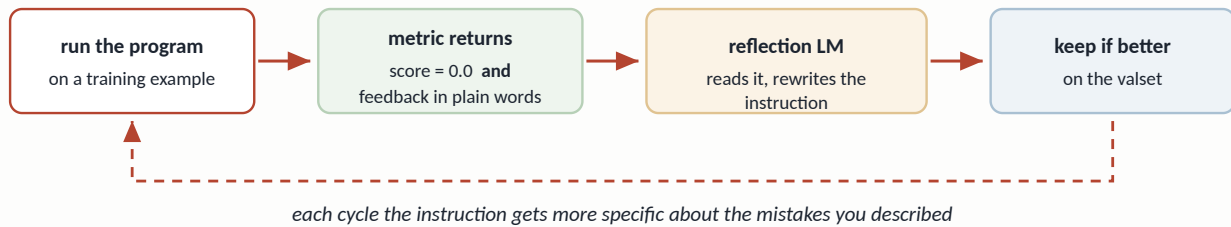


Figure 7.2 — GEPA's reflective loop. The feedback string is the whole point: it is how your domain knowledge gets into the prompt without you writing the prompt.

A GEPA METRIC WITH TEXTUAL FEEDBACK

```
import dspy

def triage_feedback(example, prediction, trace=None,
                    pred_name=None, pred_trace=None):
    if prediction.category != example.category:
        return dspy.Prediction(
            score=0.0,
            feedback=(
                f"You answered '{prediction.category}' but the correct label is "
                f"'{example.category}'. Note that this email mentions a failed charge; "
                f"anything involving money moving belongs to 'billing' even when the "
                f"customer frames it as a bug."
            ),
        )
    if abs(prediction.urgency - example.urgency) > 1:
        return dspy.Prediction(
            score=0.5,
            feedback=(
                "Category was right but urgency was off by more than one. "
                "Judge urgency by business impact, not by how upset the writer sounds."
            ),
        )
    return dspy.Prediction(score=1.0, feedback=None)

optimizer = dspy.GEPA(
    metric=triage_feedback,
    reflection_lm=dspy.LM("openai/gpt-4o", temperature=1.0, max_tokens=8000),
    auto="light",
    num_threads=8,
)

optimized = optimizer.compile(triage_program, trainset=trainset, valset=valset)
optimized.save("triage_gepa.json")
```

REFLECTION MODEL

Use your *strongest available* model as `reflection_lm`, even if the program itself runs on a small cheap one. The reflection model is called rarely (once per proposal), so it barely affects total cost — and this asymmetry is exactly how teams move a task from a frontier model down to a small local one without losing quality.

7.7 Reading what the optimizer wrote

Always look at the result. It is the most instructive artefact DSPy produces.

DIFFING THE INSTRUCTION

```
print("BEFORE:\n", triage_program.plan.predict.signature.instructions)
print("\nAFTER:\n", optimized.plan.predict.signature.instructions)
```

TYPICAL SHAPE OF THE CHANGE

BEFORE:

Sort an inbound customer support email so it reaches the right team.

AFTER:

Sort an inbound customer support email so it reaches the right team.
Assign 'billing' whenever money moves or fails to move -- failed charges, refunds, invoices, plan changes -- even if the customer describes it as a bug or an outage. Reserve 'technical' for defects with no billing impact.
Set urgency from business impact, not tone: payment failure or data loss = 5; a single user blocked with a workaround = 3; cosmetic issues = 1. Enterprise accounts raise urgency by one, to a maximum of 5.

7.8 Practical rules

Rule	Reason
Always record a baseline score first	Without it you cannot tell whether the optimizer helped or hurt.
Optimize on <code>trainset</code> / <code>valset</code> , report on <code>testset</code>	Optimizers overfit small sets enthusiastically.
Start with <code>BootstrapFewShot</code> or <code>auto="light"</code>	Cheap answer to “does this task respond to optimization at all?”
Fix the program's structure before optimizing	An optimizer cannot repair a missing retrieval step or a wrong signature.
Commit the saved <code>.json</code> to Git	It is your artefact; treat it like a lockfile, and diff it on every re-optimization.
Re-optimize when you change the model	The optimized prompt is tuned to one model's quirks; a new model deserves a new run.

Saving, Debugging and Shipping

An optimized program is only useful if you can reload it tomorrow, on another machine, and explain what it did.

8.1 Saving state (the normal way)

SAVE AND RELOAD THE LEARNED STATE

```
# --- save: a small, human-readable JSON of instructions + demos
optimized.save("trriage_v1.json")

# --- reload: rebuild the same program structure, then load state into it
program = TriageProgram()           # your class, exactly as before
program.load("trriage_v1.json")

dspy.configure(lm=dspy.LM("openai/gpt-4o-mini")) # LM config is NOT in the file
```

The file contains the optimized instructions, the selected demonstrations and signature metadata. It deliberately does **not** contain your API key, model choice or temperature — which is why the same checkpoint can be served against different models.

8.2 Saving the whole program

If the recipient will not have your Python classes, save everything:

WHOLE-PROGRAM SAVE

```
optimized.save("trriage_pkg/", save_program=True) # a directory, uses cloudpickle

loaded = dspy.load("trriage_pkg/")               # no class definitions needed
print(loaded(email=body).category)
```

PICKLE WARNING

Whole-program saves are pickles: only load ones you produced or trust, and they can break across DSPy or Python versions. Prefer the JSON state file for anything you will keep.

8.3 Debugging

Symptom	First thing to try
Output field is empty or nonsense	<code>dspy.inspect_history(n=1)</code> — look at whether the field name reads as a real instruction to the model.
Type validation keeps failing	Simplify the type (a nested Pydantic model may be too much for a small model), or move to a stronger LM for that step.
Agent calls the wrong tool	Rewrite the tool docstrings and type hints; print <code>out.trajectory</code> .

Symptom	First thing to try
Optimization made things worse	Your metric probably rewards the wrong thing. Hand-check the ten highest-scoring rows.
Results don't change when you edit code	The cache. <code>dspy.configure_cache(enable_disk_cache=False, enable_memory_cache=False)</code> .
Costs are surprising	<code>lm.history[-1]["cost"]</code> , and count how many LM calls a <code>ReAct</code> loop really makes.

TURNING ON VERBOSE LOGGING AND TRACING

```
import logging
logging.getLogger("dspy").setLevel(logging.DEBUG)

# Full step-by-step traces in a UI (optional, pip install mlflow)
import mlflow
mlflow.dspy.autolog()
mlflow.set_experiment("triage")
```

8.4 Serving it

A MINIMAL FASTAPI SERVICE

```
from fastapi import FastAPI
from pydantic import BaseModel
import dspy

dspy.configure(lm=dspy.LM("openai/gpt-4o-mini"))

program = TriageProgram()
program.load("triage_v1.json")
program = dspy.asyncify(program)  # run the sync program off the event loop

app = FastAPI()

class Ticket(BaseModel):
    email: str
    account_tier: str

@app.post("/triage")
async def triage(t: Ticket):
    out = await program(email=t.email, account_tier=t.account_tier)
    return {"category": out.category, "urgency": out.urgency}
```

STREAMING

For user-facing text, wrap the program with `dspy.streamify(...)` to get tokens as they arrive instead of waiting for the whole prediction.

8.5 A workflow that works

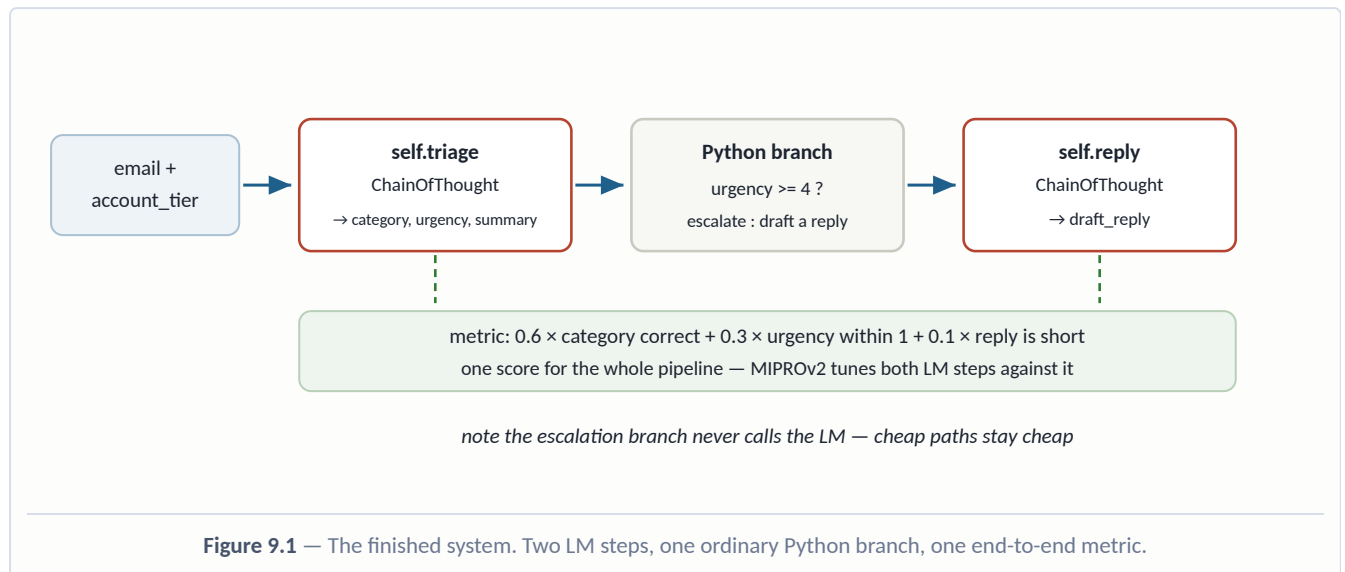
1. Write the **signature** and run it with `dspy.Predict`. Look at ten outputs.
2. Write **20 examples** and a **metric**. Run `dspy.Evaluate`. Write the number down.

3. Fix structure: add retrieval, split a step, change module type. Re-measure after each change.
4. Only now, run an **optimizer** (`auto="light"`). Compare on the test set.
5. `save()` the JSON, commit it, and record the score in the commit message.
6. Try a **cheaper model** with the same pipeline and re-optimize. This is usually where the money is.

A Complete Worked Project

Everything so far, assembled into one runnable file: a support-ticket triage system that classifies, prioritises and drafts a reply — then measures and improves itself.

9.1 What we are building



9.2 Step 1 — signatures

TRIAGE/SIGNATURES.PY

```

import dspy
from typing import Literal

Category = Literal["billing", "technical", "account", "other"]

class TriageEmail(dspy.Signature):
    """Sort an inbound customer support email so it reaches the right team.

    Judge urgency by business impact rather than the customer's tone.
    """
    email: str = dspy.InputField(desc="Raw body text of the customer's email")
    account_tier: Literal["free", "pro", "enterprise"] = dspy.InputField()

    category: Category = dspy.OutputField()
    urgency: int = dspy.OutputField(desc="1 = can wait a week, 5 = page someone now")
    summary: str = dspy.OutputField(desc="One sentence, at most 20 words")
  
```

And the second signature, for the reply we draft when the ticket is not an escalation:

TRIAGE/SIGNATURES.PY (CONTINUED)

```
class DraftReply(dspy.Signature):
    """Write a short first reply to a customer support email.

    Acknowledge the specific problem, state the next concrete step, and give
    no time estimate you cannot keep. Never promise a refund or a fix date.
    """
    email: str = dspy.InputField()
    category: Category = dspy.InputField()
    summary: str = dspy.InputField()

    draft_reply: str = dspy.OutputField(desc="Under 120 words, plain and warm")
```

9.3 Step 2 — the program

TRIAGE/PROGRAM.PY

```
import dspy
from .signatures import TriageEmail, DraftReply

class TriageProgram(dspy.Module):
    def __init__(self, escalate_at: int = 4):
        super().__init__()
        self.escalate_at = escalate_at
        self.triage = dspy.ChainOfThought(TriageEmail)
        self.reply = dspy.ChainOfThought(DraftReply)

    def forward(self, email: str, account_tier: str) -> dspy.Prediction:
        t = self.triage(email=email, account_tier=account_tier)

        # plain Python: enterprise customers are bumped one level, capped at 5
        urgency = min(5, t.urgency + 1) if account_tier == "enterprise" else t.urgency

        if urgency >= self.escalate_at:
            draft = "" # escalations are answered by a human
        else:
            draft = self.reply(email=email, category=t.category, summary=t.summary).draft_reply

        return dspy.Prediction(
            category=t.category,
            urgency=urgency,
            summary=t.summary,
            draft_reply=draft,
            escalated=urgency >= self.escalate_at,
        )
```

9.4 Step 3 — data and metric

TRIAGE/DATA.PY

```
import dspy, json, random

def load(path="tickets.jsonl", seed=0):
    rows = [json.loads(l) for l in open(path)]
    data = [
        dspy.Example(
            email=r["email"],
            account_tier=r["tier"],
            category=r["category"],      # label
            urgency=r["urgency"],        # label
        ).with_inputs("email", "account_tier")
        for r in rows
    ]
    random.Random(seed).shuffle(data)
    n = len(data)
    return data[:int(.5*n)], data[int(.5*n):int(.75*n)], data[int(.75*n):]

trainset, valset, testset = load()
print(len(trainset), len(valset), len(testset))    # 100 50 50
```

TRIAGE/METRIC.PY

```
def triage_quality(example, prediction, trace=None) -> float:
    """0.0-1.0. Category matters most, then urgency, then reply brevity."""
    cat = float(prediction.category == example.category)
    urg = float(abs(prediction.urgency - example.urgency) <= 1)
    brief = 1.0 if (prediction.escalated or len(prediction.draft_reply.split()) <= 120) else 0.0

    score = 0.6 * cat + 0.3 * urg + 0.1 * brief

    # During optimization, only near-perfect runs become demonstrations.
    if trace is not None:
        return score >= 0.9
    return score
```

9.5 Step 4 — baseline, optimize, compare

TRIAGE/RUN.PY

```
import dspy
from .program import TriageProgram
from .metric import triage_quality
from .data import trainset, valset, testset

dspy.configure(lm=dspy.LM("openai/gpt-4o-mini"))

program = TriageProgram()
evaluate = dspy.Evaluate(devset=testset, metric=triage_quality,
                        num_threads=16, display_progress=True)

# --- 1. baseline -----
baseline = evaluate(program)
print(f"baseline: {baseline.score:.1f}%")

# --- 2. optimize -----
optimizer = dspy.MIPROv2(metric=triage_quality, auto="light", num_threads=16)
optimized = optimizer.compile(program, trainset=trainset, valset=valset,
                             max_bootstrapped_demos=2, max_labeled_demos=2)

# --- 3. compare on data neither program was tuned on -----
tuned = evaluate(optimized)
print(f"optimized: {tuned.score:.1f}% (delta {tuned.score - baseline.score:+.1f})")

# --- 4. keep it -----
optimized.save("trriage_v1.json")
```

A REPRESENTATIVE RUN

```
baseline: 68.4%
optimized: 84.2% (delta +15.8)

Saved 2 optimized instructions and 4 demonstrations to triage_v1.json
```

9.6 Step 5 — use it

TRIAGE/SERVE.PY

```
import dsp
from .program import TriageProgram

dsp.configure(lm=dsp.LM("openai/gpt-4o-mini"))

program = TriageProgram()
program.load("trriage_v1.json")

out = program(
    email="Hi – you charged my card twice this morning for the same invoice "
        "(#4471). I've attached both receipts. Can you sort this out?",
    account_tier="pro",
)

print(out.category)      # billing
print(out.urgency)       # 4
print(out.escalated)     # True
print(out.summary)       # "Customer was double-charged for invoice #4471 and wants a refund."
```

9.7 What to try next on this project

Experiment	What it teaches
Swap <code>ChainOfThought</code> back to <code>Predict</code> and re-measure	Whether the reasoning field is earning its tokens on your data.
Re-run with <code>ollama_chat/llama3.2</code> and re-optimize	How much of the gap between a big and a small model optimization can close.
Replace MIPROv2 with GEPA and a feedback metric	How much your written domain knowledge is worth.
Add a <code>search_kb</code> tool and a <code>ReAct</code> reply step	Grounding replies in real documentation.
Score the draft reply with an LM judge instead of a word count	Moving from a proxy metric to a real one.

Pitfalls, FAQ and Cheat Sheet

10.1 Ten beginner mistakes

Mistake	Fix
Writing prompt text inside a signature docstring ("respond in JSON...")	Describe the task only. The adapter owns formatting.
Vague field names like <code>text</code> , <code>output</code>	Name them for what they mean; the model reads them.
Calling a module positionally: <code>qa("hi")</code>	Always keyword arguments matching the input fields.
Forgetting <code>super().__init__()</code>	Without it, optimizers see no sub-modules to tune.
Building modules inside <code>forward()</code>	Build them in <code>__init__</code> , once.
No metric, no examples, straight to optimizing	Metric first. It costs an hour and saves a week.
Optimizing and reporting on the same set	Keep a test set you touch once.
Comparing runs with the cache on	<code>dspy.configure_cache(enable_disk_cache=False, enable_memory_cache=False)</code> when measuring.
Reaching for <code>ReAct</code> when <code>Predict</code> would do	Start simple; upgrade only when a measurement demands it.
Expecting optimization to fix a broken pipeline	It tunes prompts, not architecture. Fix the structure first.

10.2 Frequently asked questions

Does DSPy fine-tune my model?

Not by default. Standard optimizers only change instruction text and few-shot demonstrations. `BootstrapFinetune` and `BetterTogether` are the exceptions and will actually train weights, which needs more data and infrastructure.

Do I have to give up my existing prompts?

No — put your best hand-written prompt in the signature docstring as the starting point. An optimizer will use it as a seed and try to beat it.

How much does optimizing cost?

With `auto="light"` on 50 examples with a small model, typically a few cents and a few minutes. `auto="heavy"` on a large program with a frontier model can reach several dollars and tens of minutes. The optimizer prints its progress; start light.

Can I use DSPy with a local model?

Yes, and it is one of the better reasons to use it. Point `dspy.LM` at Ollama, vLLM or any OpenAI-compatible endpoint. Optimization matters *more* for small models, because they benefit most from precise instructions and good demonstrations.

Is DSPy a replacement for LangChain / LlamaIndex?

They overlap but aim at different things. Those libraries are strongest at integrations — loaders, vector stores, connectors. DSPy is strongest at the prompt itself: declaring it, measuring it and improving it. Using a vector store from another library as a plain Python function inside a `dspy.Module` is completely normal.

What if my task has no right answer?

You still need *some* signal: a rubric scored by an LM judge, a preference between two outputs, or a proxy such as “did the user accept the draft”. A noisy metric is far better than none.

Do optimized prompts transfer between models?

Partially. The instructions usually transfer; the demonstrations and the fine details are tuned to one model's behaviour. Re-run the optimizer whenever you change models — it is cheap compared to rediscovering the wording by hand.

10.3 Cheat sheet

Setup

```
import dspy
lm = dspy.LM("openai/gpt-4o-mini")
dspy.configure(lm=lm)
with dspy.context(lm=other): ...
```

Signatures

```
dspy.Predict("question -> answer")
dspy.Predict("doc -> tags: list[str]")

class S(dspy.Signature):
    """Instruction goes here."""
    x: str = dspy.InputField(desc="...")
    y: int = dspy.OutputField()
```

Modules

```
dspy.Predict(sig)
dspy.ChainOfThought(sig) # + .reasoning
dspy.ReAct(sig, tools=[f], max_iters=6)
dspy.ProgramOfThought(sig)
dspy.BestOfN(module=m, N=5, reward_fn=r,
             threshold=1.0)
dspy.Refine(module=m, N=5, reward_fn=r,
            threshold=1.0)
dspy.Parallel(num_threads=16)
```

Custom program

```
class P(dspy.Module):
    def __init__(self):
        super().__init__()
        self.a = dspy.Predict("x -> y")
    def forward(self, x):
        return dspy.Prediction(y=self.a(x=x).y)
```

Data

```
ex = dsp.Example(q="...", a="...")\
    .with_inputs("q")\
    ex.inputs(); ex.labels()
```

Metric

```
def m(example, pred, trace=None) -> float:
    return float(pred.a == example.a)
```

Evaluate

```
ev = dsp.Evaluate(devset=dev, metric=m,
                  num_threads=16,
                  display_table=5)
print(ev(program).score)
```

Optimize

```
opt = dsp.MIPROv2(metric=m, auto="light")
best = opt.compile(program,
                   trainset=tr, valset=va)
```

```
opt = dsp.BootstrapFewShot(metric=m,
                           max_bootstrapped_demos=4)
```

```
opt = dsp.GEPA(metric=m_with_feedback,
               reflection_lm=big, auto="light")
```

Save / load

```
best.save("p.json")
P().load("p.json")
best.save("pkg/", save_program=True)
dsp.load("pkg/")
```

Debug

```
dsp.inspect_history(n=1)
program.named_predictors()
lm.history[-1]["cost"]
dsp.configure_cache(
    enable_disk_cache=False,
    enable_memory_cache=False)
```

10.4 Where to go next

- Rebuild the Chapter 9 project on your own data — even 30 hand-written examples work.
- Work through the official tutorials: RAG, agents, entity extraction, classification finetuning.
- Read one saved `.json` after a GEPA run end to end. It is the best writing lesson about your own task you will get.
- Try moving a working pipeline from a frontier model to a small local one, re-optimizing as you go, and measure both cost and score.

THE ONE-SENTENCE SUMMARY

Declare the task with a **signature**, choose a strategy with a **module**, define “good” with a **metric**, and let an **optimizer** write the prompt — then keep the resulting JSON in version control like any other build artefact.

Sources and Further Reading

All URLs were consulted while writing this guide, against DSPy 3.3.1.

Official documentation

1. DSPy — project home and overview.
<https://dspy.ai/>
2. DSPy — Setting up DSPy (installation, `dspy.LM`, providers).
<https://dspy.ai/getting-started/installation/>
3. DSPy — Class-based signatures.
<https://dspy.ai/getting-started/class-based-signatures/>
4. DSPy — Changing modules.
<https://dspy.ai/getting-started/changing-modules/>
5. DSPy — Composing modules.
<https://dspy.ai/getting-started/composing-modules/>
6. DSPy — Metrics.
<https://dspy.ai/getting-started/metrics/>
7. DSPy — GEPA optimization.
<https://dspy.ai/getting-started/gepa-optimization/>
8. DSPy — Saving and loading.
<https://dspy.ai/getting-started/saving-and-loading/>
9. DSPy API Reference — Modules index.
<https://dspy.ai/api/modules/>
10. DSPy API Reference — `dspy.ReAct`.
<https://dspy.ai/api/modules/ReAct/>
11. DSPy API Reference — Optimizers index.
<https://dspy.ai/api/optimizers/>
12. DSPy API Reference — `dspy.MIPROv2`.
<https://dspy.ai/api/optimizers/MIPROv2/>
13. DSPy API Reference — `dspy.BootstrapFewShot`.
<https://dspy.ai/api/optimizers/BootstrapFewShot/>
14. DSPy API Reference — `dspy.Evaluate`.
<https://dspy.ai/api/evaluation/Evaluate/>
15. DSPy API Reference — `dspy.configure_cache`.
https://dspy.ai/api/utils/configure_cache/
16. DSPy Tutorials — Retrieval-Augmented Generation.
<https://dspy.ai/tutorials/rag/>
17. DSPy Tutorials — index of all tutorials.
<https://dspy.ai/tutorials/>

18. DSPy — Choosing an optimizer.
<https://dspy.ai/diving-deeper/choosing-an-optimizer/>
19. DSPy — Cheatsheet.
<https://dspy.ai/cheatsheet/>
20. DSPy — Roadmap.
<https://dspy.ai/roadmap/>

Source code and community

21. stanfordnlp/dspy on GitHub — source, issues and release notes.
<https://github.com/stanfordnlp/dspy>
22. LiteLLM — the provider layer DSPy uses for every model call.
<https://docs.litellm.ai/>

Background reading

23. “DSPy Optimizers Explained: BootstrapFewShot, MIPROv2, COPRO and GEPA”, Future AGI.
<https://futureagi.com/blog/dspy-optimizers-explained/>
24. “Learning DSPy (3): Working with optimizers”, The Data Quarry.
<https://thedataquarry.com/blog/learning-dspy-3-working-with-optimizers/>
25. “DSPy Framework — Programmatic Prompt Optimization”, MyEngineeringPath.
<https://myengineeringpath.dev/tools/dspy-guide/>

A NOTE ON VERSION DRIFT

DSPy moves quickly and its documentation URLs are reorganised from time to time. Where a code snippet here disagrees with the library you have installed, the library wins — check `dspy.__version__` and the API reference for the class in question.